Clase 15 — DHCP, configuración de un host y Simulacro 2

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 130 min **Correlativa:** Clase 14 (sockets) · **Sigue con:** Clase 16 (UDP y multiplexión) **Material de referencia:** Tanenbaum cap. 5 (DHCP) y cap. 7 · `ACTUALIZACIONES.md` §5

🎯 Al terminar esta clase vas a poder

1. Explicar por qué un host necesita **DHCP** y qué problema resuelve frente a tener la dirección configurada a mano.
2. Describir el proceso **DORA** mensaje por mensaje, con los cuatro nombres, quién los manda, por qué son broadcast y qué lleva cada uno.
3. Justificar por qué DHCP usa **UDP** y por qué sus puertos son **67 y 68**.
4. Explicar qué es un **lease**, por qué existe y qué pasa cuando expira.
5. Explicar qué pasa cuando el DHCP **falla** y por qué existe el rango `169.254.0.0/16`.
6. Resolver un examen completo de 3 horas con cronómetro.

> **Prerequisito del examen:** `2011-02 Pregunta 2` (10 pts — DHCP con el intercambio de mensajes), `2010-03 Pregunta 4` (8 pts — agente y gestor), `2007-03 Pregunta 3` (10 pts — RARP, el antecedente de DHCP).

PARTE 1 — TEORÍA

1.1 El problema

Un host recién enchufado tiene **cero configuración de red**. No tiene:

- dirección IP
- máscara de subred
- puerta de enlace (gateway)
- servidor DNS
- nombre de dominio

Y sin esas cinco cosas **no puede hablar con nadie**. Ni siquiera puede pedir ayuda, porque pedir ayuda requiere una dirección de la que hacerlo. **Ese es el problema de arranque**, y es la razón de que DHCP exista.

Las dos salidas posibles:

Salida	Cómo	Consecuencia
Configurar a mano	un técnico va y escribe los valores	escala a **nADA**: con 500 laptops no escala
Preguntar por red	el host **pregunta** y alguien le responde	escala a toda la organización

DHCP es la segunda. La idea central es simple y es la que hay que saber:

> **Un host puede usar el Limited Broadcast para descobrir, porque el Limited Broadcast no requiere conocer la red en la que estás.**

`255.255.255.255` es una dirección de broadcast **válida en cualquier red**. Un host que no sabe nada de su subred puede mandar ahí y todos la reciben.

1.2 BOOTP, el antecedente

Antes de DHCP (RFC 951, 1984) estaba **BOOTP** (*Bootstrap Protocol*). La idea era la misma pero con una diferenciaFatal: **el servidor tenía una tabla fija** de direcciones por MAC.

```
BOOTP:  tabla en el servidor
     MAC_A → 192.168.1.25
     MAC_B → 192.168.1.26        ← estático, para siempre

DHCP:   el servidor no sabe nada
     MAC_C pide → recibe 192.168.1.27 con un lease de 24 h  ← dinámico
```

El problema de BOOTP era que **no escalaba**: un servidor tenía que tener una entrada por cada máquina, para siempre. Si una notebook se iba a casa, su dirección quedaba reservada inútilmente.

DHCP (RFC 2131, 1997) cambió dos cosas:

1. **El lease**: la dirección se presta por un tiempo y se renueva.
2. **El servidor no necesita tabla**: cualquier servidor DHCP puede atender cualquier cliente. La asignación se guarda en el cliente.

1.3 Por qué UDP y por qué los puertos 67 y 68

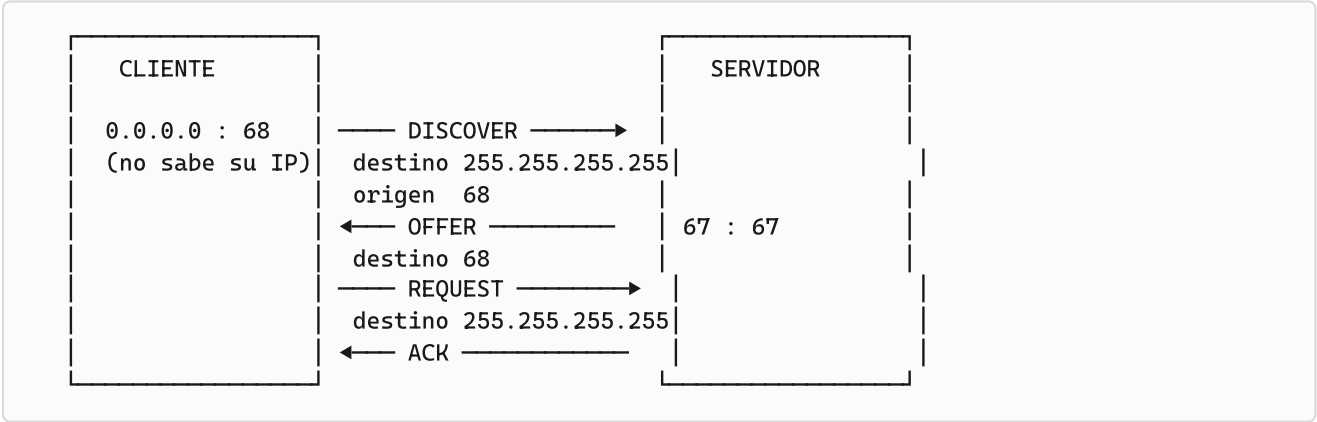
DHCP usa UDP. Es la primera pregunta que se hace, y la respuesta tiene tres partes:

1. El cliente no tiene dirección IP. No puede abrir una conexión TCP, porque abrir una conexión TCP requiere mandar un SYN **desde** alguna dirección origen. El `bind()` de un socket TCP necesita una IP local. **No la tiene.** UDP no necesita ninguna dirección previa para mandar el primer datagrama.

2. El servidor también habla desde una dirección desconocida. El cliente no sabe la IP del servidor DHCP. Y el cliente **no puede descubrir** la dirección del servidor sin ya tener una. De ahí el Limited Broadcast como destino.

3. La transacción es de un solo mensaje en cada sentido. No hay un diálogo largo que justifique el control de conexión de TCP. Y si se pierde un datagrama, el cliente simplemente **reintenta**: el protocolo tiene sus propios temporizadores y su propia lógica de reintento.

Los puertos

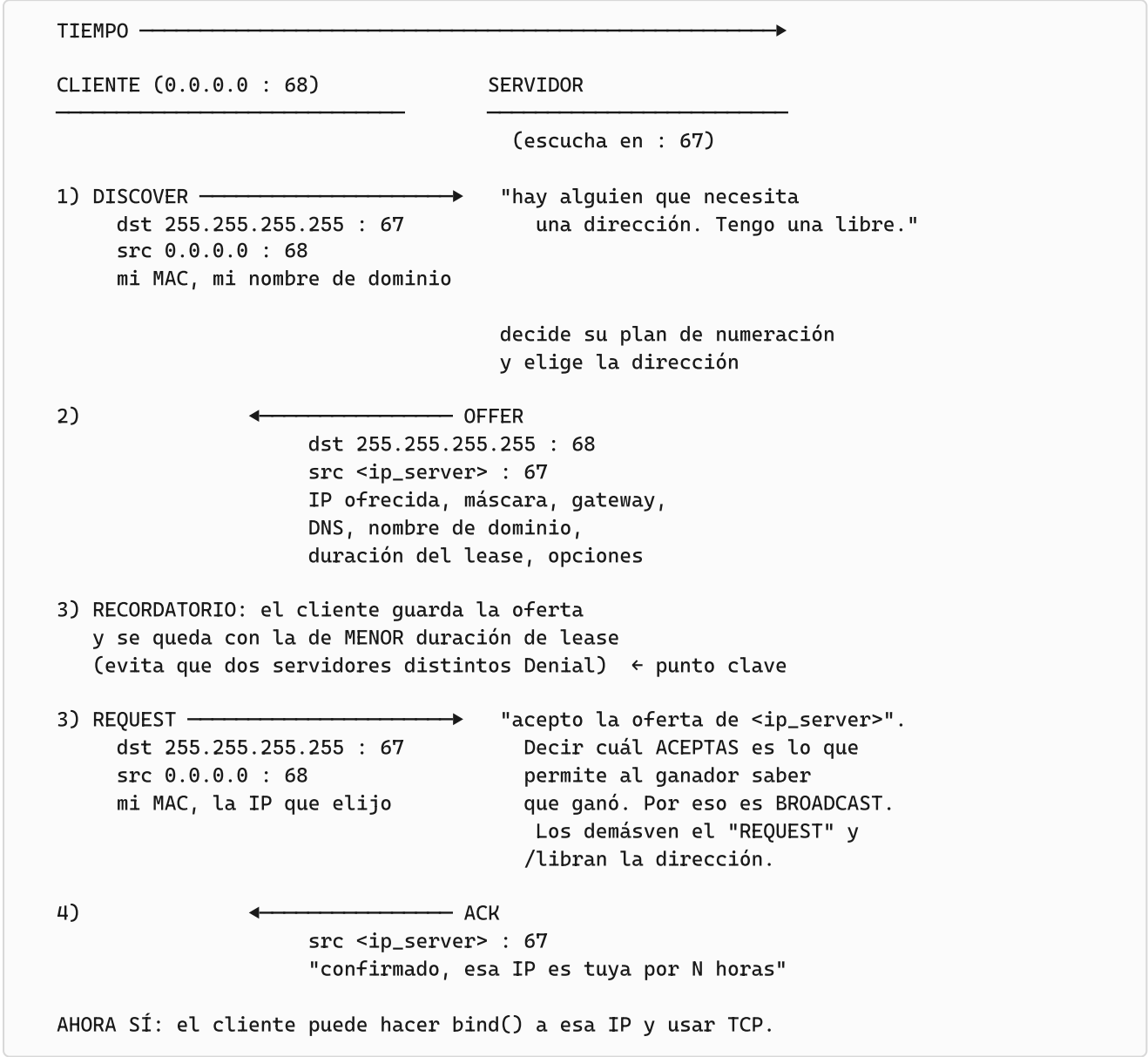


- **68** (`bootpc`) es el puerto del **cliente**. Lo elige porque el cliente **no tiene IP asignada**: el puerto 68 es una convención que funciona sin necesidad de conocer la red. Es el puerto equivalente al 1024 mínimo en TCP.
- **67** (`bootps`) es el del **servidor**, un puerto **reservado y conocido**.

> **El detalle fino que se pregunta:** en el `DISCOVER`, el campo *dirección origen IP* del datagrama IP es `0.0.0.0`, no la IP del cliente. Porque todavía no la tiene. Y el campo *dirección destino* del datagrama IP es `255.255.255.255` (Limited Broadcast), y en el datagrama UDP el puerto origen es **68**. Los tres campos juntos son la definición del "origen desconocido".

1.4 El proceso DORA, mensaje por mensaje

Las cuatro letras son **D**ISCOVER, **O**FFER, **REQUEST** y **ACK**.



Por qué el `REQUEST` es broadcast y el `OFFER` casi siempre también

Este es **el punto más importante del protocolo** y el que más se confunde:

El `REQUEST` va por broadcast obligatoriamente, aunque el cliente ya sepa la IP del servidor. ¿Por qué? Porque si fuera unicast, cada servidor sabría "me eligieron a mí" o "no me eligieron", y el que **no** fue elegido sabría que otro servidor le ganó. Eso es exactamente lo que se quiere evitar: que un cliente que eligió el precio más bajo (como en el problema de `connectCheaper()` de la clase 14) no se lo anuncie a los demás. El broadcast mantiene el secreto delWinner.

El `OFFER`, en cambio, se puede mandar por unicast si el servidor ya tiene la MAC del cliente (y a veces la tiene, de un `REQUEST` anterior). **Many DHCP implementations mandan OFFER por broadcast**, porque el cliente recién arrancando puede no tener todavía su ARP resuelto. Es una decisión de implementación.

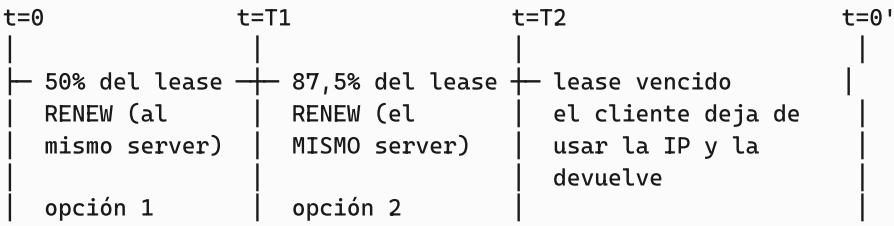
Los 4 mensajes en detalle

#	Mensaje	De → a	IP origen → destino	Puerto UDP	Contenido clave
1	**DISCOVER**	cliente → todos	`0.0.0.0` → `255.255.255.255`	68 → 67	MAC del cliente, nombre de dominio
2	**OFFER**	servidor → cliente	ip_server → `255.255.255.255`	67 → 68	IP, máscara, gateway, DNS, lease
3	**REQUEST**	cliente → todos	`0.0.0.0` → `255.255.255.255`	68 → 67	qué servidor eligió
4	**ACK**	servidor → cliente	ip_server → `255.255.255.255`	67 → 68	confirmación final

Todos van por `255.255.255.255` (Limited Broadcast) menos el `OFFER`, que puede ir por unicast. Es la única manera de que funcione sin conocer la red.

1.5 El lease: qué es y por qué existe

El **lease** es el **período de tiempo** durante el cual la asignación es válida.



El cliente tiene **dos mecanismos de renovación**:

- **T1 = 50 %** del lease: manda `REQUEST` **al mismo servidor** que le dio la IP. Es una Optimization: si funciona, no hay nada que cambiar.
- **T2 = 87,5 %** del lease: si el primero falló, manda `REQUEST` a **cualquier servidor** (por broadcast), pidiendo cualquiera.
- **Al vencer**: el cliente deja de usar la IP. **Y no la devuelve**: simplemente deja de renovar. El servidor la vuelve a poner en el pool cuando expire su registro interno.

¿Por qué existe el lease? Porque un host puede **desaparecer sin avisar**: se apaga, se rompe, se va de viaje. Si la asignación fuera fija (BOOTP), esa dirección quedaría **reservada para siempre** para un equipo que no existe. Con lease, el problema se resuelve solo.

1.6 Opciones DHCP que hay que saber

El `OFFER` y el `ACK` llevan un campo **options** con pares (código, longitud, valor). Los que se preguntan:

Opción	Código	Qué le dice al cliente
Subnet Mask	1	la máscara, para que sepa qué es su red local
Router (gateway)	3	a quién le pregunta por destinos fuera de su red
Domain Name Server	6	la IP del resolver DNS
Hostname	12	el nombre que el cliente quiere tener
Domain Name	15	el dominio local
Requested IP Address	50	el cliente pide una IP concreta (arranque de un equipo específico)
IP Address Lease Time	51	la duración del lease, en **segundos**
Message Type	53	1=DISCOVER, 2=OFFER, 3=REQUEST, 5=ACK
Server Identifier	54	**qué servidor** hizo la OFFER (clave para el REQUEST)
Parameter Request List	55	qué opciones quiere el cliente
Renewal (T1)	58	el 50 % del lease
Rebinding (T2)	59	el 87,5 % del lease

> **El par (53, 54) es el corazón del protocolo.** El campo *Message Type* dice **qué** mensaje es. Y el *Server Identifier* dice **quién** lo mandó, que es lo que permite que el `REQUEST` del cliente diga "acepto la de **ese** servidor".

Los tres pasos del TFTP

DHCP no es un protocolo de sesión. **Cada mensaje es un mensaje TFTP** (*Trivial File Transfer Protocol*), que funciona sobre UDP. Los pasos:

```
1. REQUEST   el cliente pide su configuración
2. OFFER     el servidor ofrece una dirección
3. ACK       el servidor confirma

(y el DISCOVER es un REQUEST con Message Type = DISCOVER)
```

TFTP existe porque se necesitaba **algo muy simple** para que un cliente que no tiene IP pudiera hablar con un servidor. No tiene ni número de secuencia ni confirmaciones: **DHCP se encarga de reintentar** si algo se pierde, con sus propios temporizadores.

1.7 Qué pasa si el DHCP falla

Si el cliente no recibe ninguna `OFFER` (no hay servidor DHCP en la red), hace lo que hace **todo** protocolo: **reintenta**. Periódicamente manda un nuevo `DISCOVER`.

Pero si la red **no tiene ningún DHCP**, el host queda sin dirección. Y como no puede pedir nada, se autoconfigura:

```
┌──────────────────────────────────────────────────────┐
│  APIPA  (Automatic Private IP Addressing)            │
│  Microsoft inventó el nombre; el nombre real es      │
│  "link-local" y el rango es 169.254.0.0/16           │
└──────────────────────────────────────────────────────┘

El cliente:
  1. se elige una IP al azar dentro de 169.254.1.0 – 169.254.254.255
     (o prueba una y hace ARP para ver si nadie responde)
  2. pone su máscara como /16
  3. NO tiene gateway  →  no puede salir de la red local
  4. no tiene DNS      →  no resuelve nombres
```

¿Por qué `169.254.0.0/16` y no otra cosa? Porque la IANA lo reservó específicamente para esto. Y hay una razón de diseño más profunda:

> **Es un rango que nunca se va a rutear en Internet.** Aunque por un error de configuración un paquete con destino `169.254.x.x` llegara a Internet, ningún router lo va a reenviar. O sea: **si la autoconfiguración falla, el host no puede molestar a nadie.** Es un rango "atrapado" por diseño.

Es un detalle de diseño elegante que vale la pena conocer: **el rango de autoconfiguración es elegido para que sea inofensivo.**

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: - El proceso **DORA** completo, los 4 mensajes, en ese orden. - **UDP**, puertos **67** y **68**, y el Limited Broadcast `255.255.255.255`. - El `DISCOVER` con IP origen `0.0.0.0`. - El **`REQUEST` por broadcast** y la razón de ser (el secreto de qué servidor ganó). - El lease con **T1 = 50 %** y **T2 = 87,5 %**, y los dos mecanismos de renovación. - **`169.254.0.0/16`** como rango link-local de autoconfiguración, no ruteable. - Los campos **Message Type** y **Server Identifier** como el corazón del protocolo.

Cambió: - **DHCPv6** existe y usa **UDP 546 (cliente) y 547 (servidor)**, con **multicast** en vez de broadcast. No es "el DHCP de IPv6": es un protocolo distinto, porque en IPv6 el cliente **ya tiene una dirección link-local derivable del MAC** y no necesita el Limited Broadcast para nada. - **IPv6 privacy extensions y SLAAC**: en IPv6 la autoconfiguración es **nativa** (`RFC 4862`) y no depende de ningún servidor DHCPv6. El "SLAAC" genera la dirección a partir del prefijo que anuncia el router en los RA. - **La era del CGNAT** (Carrier-Grade NAT, que grew con el agotamiento de IPv4 en RIPE en noviembre de 2019) cambió el contexto: los ISP dan a los usuarios direcciones **compartidas**, lo cual hace que algunos servicios de TV por IP y VoIP necesiten otras soluciones. No afecta al DHCP del cliente final, pero explica por qué la capa de acceso a Internet se ve distinta hoy.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La traza DORA completa con direcciones concretas

Datos. Red `192.168.1.0/24`. Gateway `192.168.1.1`. DNS `132.42.120.5`. Servidor DHCP `192.168.1.10`. Lease de 24 horas. El cliente es `PC` con MAC `00:1A:2B:3C:4D:5E`.

Resolución, mensaje por mensaje, con la tabla completa:

#	Mensaje	IP orig → dest	UDP orig → dest	MAC orig → dest	Contenido
1	**DISCOVER**	`0.0.0.0` → `255.255.255.255`	68 → 67	PC → `FF:FF:FF:FF:FF:FF`	MAC, hostname
2	**OFFER**	`192.168.1.10` → `255.255.255.255`	67 → 68	srv → broadcast	IP `192.168.1.50`, /24, GW `.1`, DNS `132.42.120.5`, lease 86400 s
3	**REQUEST**	`0.0.0.0` → `255.255.255.255`	68 → 67	PC → broadcast	Server ID = `192.168.1.10`, pide `192.168.1.50`
4	**ACK**	`192.168.1.10` → `255.255.255.255`	67 → 68	srv → broadcast	confirma `192.168.1.50`

Verificación: ¿qué haría PC si el `ACK` no llegara? Lost el lease no está confirmado, así que **no puede usar la dirección**. Reintenta el `REQUEST`. Si no hay respuesta tras N intentos, vuelve al paso 1 (`DISCOVER`).

Verificación 2: ¿qué pasa si el `OFFER` del servidor se pierde? PC no lo sabe. Reintenta el `DISCOVER`. El servidor ve un `DISCOVER` nuevo y hace una `OFFER` nueva. **No hay conflicto**, porque el servidor todavía no registró nada: el registro se hace recién cuando recibe el `REQUEST`.

2.2 Ejercicio 2 — Los temporizadores del lease

Datos. El lease es de **24 horas** = 86 400 segundos.

Resolución:

```
T1 = 0,50  × 86 400 =  43 200 s = 12 h
T2 = 0,875 × 86 400 =  75 600 s = 21 h
vencimiento =          86 400 s = 24 h
```

Interpretación: el cliente intenta renovar con el mismo servidor a las 12 h. Si en los próximos 9,6 h no obtiene respuesta, a las 21 h pregunta a **cualquier** servidor. Si a las 24 h no tiene nada, deja de usar la dirección.

¿Por qué T2 es 87,5 % y no 90 % o 95 %? Es un compromiso: si el umbral fuera muy tardío, no quedaría tiempo para conseguir una dirección nueva antes de que venza la actual. El 12,5 % restante (3 horas en

este caso) es el margen para que el `REQUEST` de rebinding llegue, el servidor responda y el cliente configure la dirección nueva.

2.3 Ejercicio 3 — Cuántas direcciones necesita la red

Datos. Una empresa quiere 300 empleados, 20 servidores y 5 impresoras en una sola LAN con DHCP.

Resolución — el cálculo del pool:

```
Total de dispositivos que piden IP:  300 + 20 + 5 = 325
+ 1 para el gateway                       = 326
+ 1 para el servidor DHCP                  = 327
+ 1 para la red y 1 para el broadcast      = 329

2^9 − 2 = 512 − 2 = 510  >= 327   ✓    →  necesitan un /23
```

Ahora la pregunta de diseño, que es la que seReward en el examen: ¿cuántas direcciones **quedan libres**?

```
Con un /23 (512 direcciones):     512 − 329 = 183 libres
Con un /22 (1024 direcciones):   1024 − 329 = 695 libres
```

El criterio correcto es el /23, y la razón de por qué es la que hay que saber decir: **un plan de numeración se dimensiona para crecer, no para el tamaño de hoy.** Con un /23 tenés 183 direcciones de margen, que es un 56 % de crecimiento sobre la necesidad actual. Con un /22 tendrías el doble de freedom y no lo usás.

> **Y esa es exactamente la lógica del apartado d del Problema 2 del Examen 2011-03** (resuelto en la Clase 27 y la 29): "*dejar la mayor cantidad posible de direcciones IP para uso futuro*" **no** significa usar el bloque entero, sino usar el **menor bloque que cubra la necesidad** y reservar el resto.

2.4 Ejercicio 4 — Dos servidores DHCP

Datos. En la red hay dos servidores DHCP, `A` (192.168.1.10) y `B` (192.168.1.11). Llegan dos clientes.

Resolución — el paso que casi nadie explica, el paso 3:

```
t=0  PC1 ──DISCOVER (broadcast)──►  A y B lo reciben

t=1  A ──OFFER: 192.168.1.50, lease 24 h ──►  PC1
    B ──OFFER: 192.168.1.60, lease 48 h ──►  PC1

PC1 aplica el DESEMPATE DE TIE-BREAK:
    "el que ofrezca el lease MÁS CORTO"

    A ofrece 24 h  →  gana A  ✗ descartada la de B
    B ofrece 48 h  →  pierde

t=2  PC1 ──REQUEST (broadcast)──►  "el Server Identifier va en 192.168.1.10,
                                      y elijo la 192.168.1.50"
                           ──►  A  lee:  soy yo  →  registro la asignación
                           ──►  B  lee:  no soy yo  →  LIBERO 192.168.1.60

t=3  A ──ACK: 192.168.1.50 es tuya por 24 h ──►  PC1
```

El punto clave: el `REQUEST` **no es unicast**, y por eso B puede saber que perdió y liberar la dirección que había reservado. **Si fuera unicast**, B seguiría creyendo que tiene la dirección asignada hasta que venciera el lease.

Y el **tie-break de lease más corto** parece contraintuitivo (¿por qué esperar *menos*?) pero tiene una razón clara: **minimiza el tiempo en que un cliente queda "atrapado" con una dirección de un servidor caído**. Si un cliente aceptara la oferta del servidor que no está disponible, tardaría 48 h en poder usar otra. Con el desempate por lease corto, y el T2 al 87,5 %, recupera la conectividad en unas horas.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Pregunta 2 (10 puntos) — DHCP

> **Fuente:** Examen `eirc1102` / `sirc1102`, 3 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf

Enunciado (texto del examen)

> Explique el funcionamiento del protocolo DHCP detallando el intercambio de mensajes entre cliente y servidor.

La solución que hay que escribir

Punto 1 — Qué resuelve (2 pts). DHCP es un protocolo de **arranque** (*bootstrap*) que permite a un host **sin configuración** obtener su dirección IP, máscara, gateway y DNS automáticamente. Sin él, cada equipo nuevo requeriría que un técnico lo configurara a mano.

Punto 2 — Por qué UDP (2 pts). Porque el cliente **no tiene todavía dirección IP**, y no puede abrir una conexión TCP (el `SYN` necesita una dirección origen válida). La transacción son mensajes sueltos de un solo envío, sin necesidad de estado de conexión.

Punto 3 — DORA (4 pts). Los cuatro mensajes, con quién los manda y qué contienen:

Paso	Mensaje	Remitente	Destinatario	Contenido
1	**DISCOVER**	cliente	broadcast `255.255.255.255`	identifica al cliente (MAC, hostname) y pide dirección
2	**OFFER**	servidor	cliente (broadcast o unicast)	propone una dirección libre, con máscara, gateway, DNS, duración del lease y demás opciones
3	**REQUEST**	cliente	broadcast	**confirma cuál de las ofertas acepta**, identificando al servidor elegido
4	**ACK**	servidor	cliente	confirma la asignación y fija la duración del lease

Punto 4 — El detalle que suma (2 pts). El **REQUEST va por broadcast** aunque el cliente ya sepa la dirección del servidor, y **por eso el paso 3 es obligatorio**: es lo que le permite a los servidores que hicieron OFFER y **no** fueron elegidos saber que perdieron y liberar la dirección que habían reservado. Si fuera unicast, cada uno creería que ganó.

Los tres pasos de TFTP. Los DHCP se apoya en TFTP (*Trivial File Transfer Protocol*) sobre UDP, por la misma razón: tiene que funcionar desde un host sin dirección. TFTP es trivial justamente porque **DHCP se encarga de manejar los tiempos y los reintentos** que TFTP no tiene.

El lease y los temporizadores. La asignación **no es permanente** sino que tiene una duración. El cliente intenta renovarla cuando se ha consumido el 50 % del lease (T1), contra el mismo servidor. Si no logra respuesta, al 87,5 % (T2) reintenta contra cualquier servidor. Si el lease se vence sin renovación, el cliente libera la dirección y vuelve a solicitar una.

Complemento — RARP, el antecedente que se pregunta en otro examen

> **Examen 2007-03, Pregunta 3 (10 pts)**: *Describa en qué escenario se utiliza el protocolo RARP y su funcionamiento.*

La respuesta corta: RARP (`RFC 826`) es **el antecesor de DHCP**. Antes de que existiera el boot de un host sin disco duro, un host que encendía sin sistema operativo instalado necesitaba **descubrir su propia dirección IP**. No sabía cuál era, y sin ella no podía hablar con nadie.

ARP responde: "*¿qué MAC tiene esta IP?*". **RARP invierte la pregunta**: "*¿qué IP tiene este MAC?*". El host manda un broadcast `¿quién es 00:1A:2B:3C:4D:5E?` y el servidor de red le responde con una dirección.

```
ARP   :  "¿qué MAC corresponde a la IP 192.168.1.50?"     → el router responde
RARP  :  "¿qué IP corresponde al MAC 00:1A:2B:3C:4D:5E?"  → el host pregunta
```

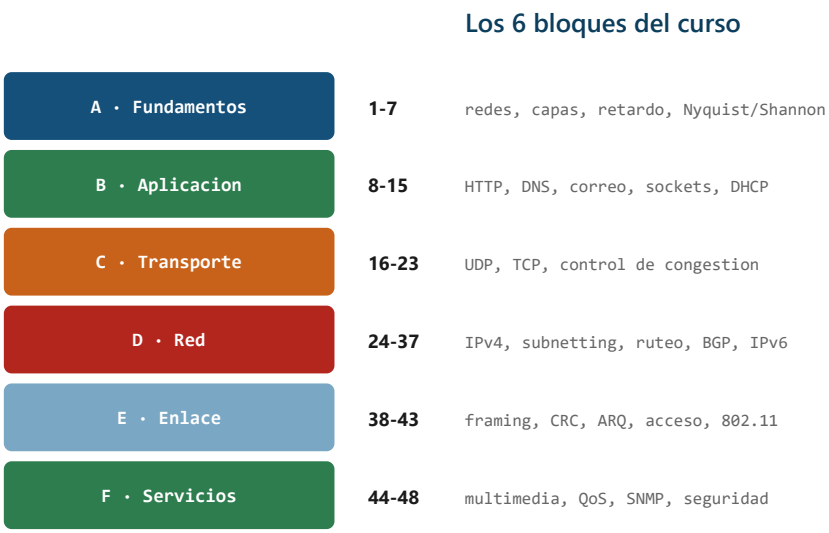
Por qué ya no se usa:

Problema de RARP	Lo resuelve DHCP
Solo devuelve **una** dirección IP. Nada de máscara, ni gateway, ni DNS	devuelve **toda** la configuración
No hay lease: la asignación es fija para siempre	hay lease y renovación
Necesita un **servidor RARP** en cada red	un solo servidor DHCP para toda la organización
No existe para IPv6	DHCPv6, y además SLAAC

La respuesta completa al examen 2007-03 (10 pts): escenario, formato de los mensajes (idéntico a ARP pero con tipo de operación distinto: RARP usa los tipos 3 y 4 donde ARP usa 1 y 2), la tabla del servidor, y **las tres limitaciones** de la tabla de arriba. Mencionar que DHCP **supera** a RARP en todo menos en complexidad.

PARTE 4 — PARTE B DEL SIMULACRO 2

4.1 Repaso del bloque B (clases 8 a 15)



El mapa en una línea por clase:

```
8  HTTP/1.1: petición/respuesta, persistente vs. no persistente
9  HTTP: cabeceras, caché con ETag, cookies, HTTP/2 y HTTP/3
10 DNS: espacio jerárquico + base distribuida, RR de 5 campos, tipos de registro
11 DNS: iterativo vs. recursivo, caché y TTL, DoH y DNSSEC
12 Taller: resolución completa paso a paso, RTT de una página
13 Correo: MUA/MTA/MDA, SMTP push, POP3 e IMAP pull, MIME, dot-stuffing
14 Sockets: API de Berkeley, orden de llamadas, htons, errores del código oficial
15 DHCP: DORA, lease, 169.254/16, Simulacro
```

Las tablas que hay que saber de memoria:

Resource Record, 5 campos:

```
nombre   | clase | tipo | TTL | datos
2 bytes | 2 B   | 2 B  | 4 B | variable
```

Tipos de RR que hay que saber: A, AAAA, NS, CNAME, MX, SOA, PTR, TXT, SRV, CAA.

DORA:

```
D  DISCOVER  cliente → broadcast   0.0.0.0:68 → 255.255.255.255:67
O  OFFER     servidor → cliente    67 → 68    (propone IP, máscara, GW, DNS, lease)
R  REQUEST   cliente → broadcast   "acepto la oferta del servidor X"
A  ACK       servidor → cliente    confirma
```

Puertos que hay que saber:

```
20/21 FTP       22 SSH       25 SMTP       53 DNS       67/68 DHCP
80 HTTP         110 POP3     143 IMAP      443 HTTPS    546/547 DHCPv6
179 BGP         853 DoT      161/162 SNMP
```

Petición/respuesta vs. push/pull:

Protocolo	Modelo	Quién inicia	Ejemplo
HTTP	**petición/respuesta**	el cliente	8, 9
DNS	**petición/respuesta**	el cliente	10, 11
SMTP	**push**	el emisor	13
POP3 / IMAP	**pull**	el receptor	13
DHCP	**petición/respuesta** (pero por broadcast)	el cliente	15

4.2 Los 8 errores más frecuentes del bloque

#	El error	Lo correcto	Por qué
1	"DNS usa UDP y si es grande pasa a **otro puerto**"	pasa a **TCP en el 53**	es el mismo puerto, distinto protocolo
2	"El **TLD** devuelve la IP del dominio"	devuelve el **NS autoritativo**	el TLD no conoce los dominios que cuelgan de él
3	"El **REQUEST** de DHCP va por unicast porque ya conoce al servidor"	va por **broadcast**	para que los perdedores liberen la dirección reservada
4	"El **DISCOVER** de DHCP tiene como origen la MAC del cliente"	origen IP es `0.0.0.0`	no tiene IP todavía; la MAC va en un campo de opción
5	"El **ACK** de TCP es un paquete"	es un **segmento con el flag ACK**	el ACK va piggybacked en el segmento de datos del otro sentido
6	"POP3 borra los mensajes al hacer `DELE` "	`DELE` **solo marca**, borra en `QUIT`	para poder deshacer si algo sale mal
7	"Para que haya confidencialidad hace falta **cifrar**"	para **integridad** alcanza un **hash**	el hash detecta el cambio; el cifrado lo esconde
8	"El cliente hace `bind()` antes de `connect()` "	el cliente **no hace bind**	el servidor reserva un puerto conocido; el cliente usa uno efímero

4.3 📝 Simulacro 2 — Examen completo

> **Instrucciones del examen real:** 3 horas · sin material · individual · **aprobación con 60/100** · **obligatorio responder correctamente al menos 15 puntos en las preguntas teóricas**.

Enunciado — Preguntas teóricas

Teórica 1 (10 puntos). Un usuario abre su navegador y escribe `https://www.ejemplo.com/index.html`. Describa, en orden cronológico, **todos** los pasos que ocurren desde que aprieta Enter hasta que ve la página en pantalla. Sea específico sobre: qué protocolos participan, cuántos *round trips* aproximados implica cada uno, y qué pasa si alguno de ellos falla.

Teórica 2 (10 puntos). Describa el funcionamiento del protocolo **DNS**, explicando el papel de cada tipo de servidor y la diferencia entre los dos algoritmos de resolución (iterativo y recursivo). Explique además el funcionamiento del mecanismo de **caché**, incluyendo qué campo controla el tiempo que un registro permanece en caché y quién lo define.

Teórica 3 (8 puntos). Describa el proceso de **arranque de un host** en una red con DHCP, detallando los mensajes involved. Explique por qué el protocolo usa UDP y no TCP, y qué ocurre si el servidor DHCP no está disponible.

Teórica 4 (8 puntos). Compare los protocolos **POP3** e **IMAP**, y explique el problema que surge cuando un mismo canal transmite información de control y datos simultáneamente, y cómo lo resuelve el protocolo.

Teórica 5 (4 puntos). Mencione tres condiciones que deben cumplirse para que una conexión **TCP** pueda establecerse entre dos procesos, e indique qué función de la API de sockets debe usar cada extremo para empezar el proceso.

Enunciado — Problemas prácticos

Problema 1 (30 puntos). Una organización tiene un único servidor DHCP que atiende dos redes separadas por un router:

- **Red A** (`192.168.10.0/24`): 120 puestos de trabajo y 2 servidores.
- **Red B** (`10.20.30.0/24`): 40 puestos de trabajo y 1 servidor con direccionamiento estático (no usa DHCP).

Se pide:

a) Especifique los **parámetros que debe entregar el servidor DHCP** a cada tipo de cliente de la red A y de la red B. Para cada parámetro indique el **valor** que corresponde según los datos del problema.

b) Realice la **traza completa del proceso de arranque** de un puesto de trabajo de la red A, indicando para cada uno de los cuatro mensajes: emisor, receptor, direcciones IP de origen y destino (de nivel IP y de nivel UDP), y qué información transporta.

c) El router entre las dos redes **no permite tráfico de la red B hacia la red A** salvo las respuestas a soliciting DHCP. Explique qué problema genera esto para los clientes de la red B que attempting renueva su lease, y proponga una solución.

Problema 2 (30 puntos). Se desea implementar un servicio de resolución de nombres simplified para uso interno. El servicio expone la función:

```
string consultar(string nombreHost, int tipoConsulta, int clase)
```

que devuelve, si el nombre existe y hay registros del tipo pedido, la **cadena de valores** separada por comas; y devuelve `""` si el nombre no existe.

Se cuenta con las siguientes primitivas:

- `enviar_consulta(servidorDNS, nombreHost, tipoConsulta, clase)` — envía una consulta DNS al servidor indicado.
- `esperar_respuesta()` — bloquea hasta recibir una respuesta. Devuelve un registro con los campos: `nombre`, `tipo`, `clase`, `ttl`, `datos`, y `codigoError`. Devuelve `null` si no hay respuesta (timeout).
- `hayMasRegistros()` / `obtenerSiguienteRegistro()` — permiten recorrer los registros de la respuesta.

a) Explique qué información debe contener el mensaje de consulta DNS que la función debe construir, y por qué.

b) Implemente la función `consultar` en un lenguaje de alto nivel, teniendo en cuenta que: (i) el nombre puede no existir, (ii) un nombre puede tener **varios** registros del mismo tipo, (iii) el servidor puede **no responder**, y (iv) el servidor puede responder con un **código de error** que no sea "no existe".

c) Explique qué consecuencia tiene, en la resolución real, el hecho de que su implementación **no pueda** distinguir entre "el nombre no existe" y "el servidor no respondió", y cómo resolvería ese defecto.

Cronómetro sugerido

Momento	Minutos	Qué hacer
0:00 – 0:08	8	leer todo, elegir qué problemas hacer
0:08 – 0:53	45	**las 5 teóricas** (8 a 10 min cada una)
0:53 – 1:53	60	**Problema 1**
1:53 – 2:53	60	**Problema 2**
2:53 – 3:00	7	releer, corregir, completar las cotas

> **Recordá el mínimo de 15 puntos en las teóricas.** Si te quedás sin tiempo para los problemas, todavía podés aprobar: **no abandones las teóricas**.

▶ ✅ **RESPUESTAS DEL SIMULACRO 2**

🔗 Referencias

- **Examen** `eirc1102` **/** `sirc1102` — Pregunta 2 completa (10 pts, DHCP)
- **Examen** `eirc0703` **/** `sirc0703` — Pregunta 3 (10 pts, RARP y su relación con DHCP)
- **Examen** `eirc1012` **/** `sirc1012` — Problema 1 (30 pts, sockets UDP)
- **Examen** `eirc1003` **/** `sirc1003` — Pregunta 2 (8 pts, arquitectura de correo), P4 (8 pts, SNMP)
- **Examen** `eirc0812` **/** `sirc0812` — Pregunta 4 (7 pts, POP3 y CRLF.CRLF)

- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 (DHCP) y cap. 7
- **RFC 2131** (DHCP) · **RFC 2132** (opciones DHCP) · **RFC 826** (RARP)
- `ACTUALIZACIONES.md` **§5** — DHCPv6, SLAAC, CGNAT
- **Clase siguiente:** 16 — Multiplexación, demultiplexación y UDP